1. Introduction to C Programming

---

1. History and Importance of C

What is C?

C is a **programming language** used to talk to computers and give instructions.

Who created C?

- Created by
- Developed in **1972**
- At

Why is C Important?

Think of C like **basic foundation**:

- 🏗️ Many languages (C++, Java, Python) are based on C
- ⚡ Very fast and powerful
- 💻 Used to build:
  - Operating Systems (like Windows, Linux)
  - Games
  - Embedded systems (machines, devices)

👉 Simple idea:
If you learn C, learning other languages becomes easy.

---

2. Structure of a C Program

A C program always follows a **fixed structure (format)**.

🧩 Basic Structure:

```
#include <stdio.h>

int main() {
   printf("Hello World");
   return 0;
}
```

🔍 Explanation (Very Simple):

Part	Meaning
#include <stdio.h>	Tells computer to use input/output tools
int main()	Starting point of program
{ }	Body of program
printf()	Used to print output
return 0;	End of program

🧠 Easy Understanding:

👉 Program = Recipe
👉 Computer = Cook
👉 Code = Instructions

---

3. Writing and Executing First C Program

✍️ Step 1: Write Code

```
#include <stdio.h>

int main() {
   printf("Hello, I am learning C!");
   return 0;
}
```

▶️ Step 2: Compile (Convert to machine language)

👉 Using compiler (like Turbo C, GCC)

▶️ Step 3: Run

👉 Output will be:

```
Hello, I am learning C!
```

🧠 Simple Analogy:

- Writing code = Writing English sentence
- Compiler = Translator

- Output = Final result on screen

💬 4. Comments in C

📖 What is a Comment?

👉 Comments are **notes for humans**, not for computer.

Computer will **ignore comments**.

✍️ Types of Comments

🟢 Single-line comment

```
#include <stdio.h>

int main() {
   // This prints a message
   printf("Hello");
   return 0;
}
```

🔵 Multi-line comment

```
#include <stdio.h>

int main() {
   /* This is
      a multi-line
      comment */
   printf("Hello");
   return 0;
}
```

Data Types and Variables in C

🔢 5. Basic Data Types in C

📖 What is a Data Type?

👉 Data type tells the computer **what kind of data you are storing**

🧩 Main Data Types:

Data Type	Meaning	Example
`int`	Integer (whole numbers)	10, 25, -5
`float`	Decimal numbers	3.5, 6.78
`char`	Single character	'A', 'b'
`double`	Big decimal (more precision)	3.141592

🧠 Easy Example:

```
int age = 18;
float marks = 89.5;
char grade = 'A';
double pi = 3.14159;
```

👉 Think:

- `int` → counting numbers
- `float` → normal decimals
- `double` → more accurate decimals
- `char` → letters

📦 6. Declaring and Initializing Variables

📖 What is a Variable?

👉 Variable = **container to store data**

✍️ Declaration (creating variable)

```
int age;
float marks;
char grade;
```

👉 You are telling computer: “Hey! I need space to store this type of data”

✍️ Initialization (giving value)

```
int age = 18;
float marks = 95.5;
char grade = 'A';
```

🧠 Real-Life Example:

👉 Variable = Water bottle
👉 Value = Water inside

🧪 Full Example Program

```
#include <stdio.h>

int main() {
    int age = 18;
    float marks = 92.5;
    char grade = 'A';

    printf("Age: %d\n", age);
    printf("Marks: %f\n", marks);
    printf("Grade: %c\n", grade);

    return 0;
}
```

🔍 Output:

```
Age: 18
Marks: 92.500000
Grade: A
```

🔒 7. Constants

📖 What is a Constant?

👉 Constant = **value that cannot change**

✍️ Example:

```
const int x = 10;
```

👉 You **cannot change x later**

❌ Wrong:

```
x = 20;   // ERROR ❌
```

🧠 Real-Life Example:

👉 Your date of birth → constant (never changes)

🧪 Program Example:

```
#include <stdio.h>

int main() {
    const float pi = 3.14;

    printf("Value of PI: %f", pi);

    return 0;
}
```

📥📤 8. Input and Output (scanf & printf)

📤 `printf()` → Output

👉 Used to **display data**

```
printf("Hello World");
```

📥 `scanf()` → Input

👉 Used to **take input from user**

🧩 Format Specifiers:

Data Type	Symbol
int	`%d`
float	`%f`
char	`%c`
double	`%lf`

🧪 Example Program:

```
#include <stdio.h>

int main() {
    int age;
    float marks;

    printf("Enter your age: ");
    scanf("%d", &age);

    printf("Enter your marks: ");
    scanf("%f", &marks);

    printf("Your age is: %d\n", age);
    printf("Your marks are: %f", marks);

    return 0;
}
```

🔍 Output Example:

```
Enter your age: 18
Enter your marks: 90.5
Your age is: 18
Your marks are: 90.500000
```

⚠️ Important Point:

👉 In `scanf`, we use **& (address symbol)**

Example:

```
scanf("%d", &age);
```

Operators and Expressions in C

👉 **Operator** = symbol that performs an action
👉 **Expression** = combination of values + operators

Example:

```
int sum = 5 + 3;   // expression
```

➕ 9. Arithmetic Operators

📖 Used for calculations

Operator	Meaning	Example
`+`	Addition	5 + 3 = 8
`-`	Subtraction	5 - 3 = 2
`*`	Multiplication	5 * 3 = 15
`/`	Division	10 / 2 = 5
`%`	Modulus (remainder)	10 % 3 = 1

🧪 Example Program:

```
#include <stdio.h>

int main() {
    int a = 10, b = 3;

    printf("Add: %d\n", a + b);
    printf("Sub: %d\n", a - b);
    printf("Mul: %d\n", a * b);
    printf("Div: %d\n", a / b);
    printf("Mod: %d\n", a % b);

    return 0;
}
```

⚖️ 10. Relational Operators

📖 Used for comparison (True/False)

Operator	Meaning
`==`	Equal to
`!=`	Not equal

Operator	Meaning
>	Greater than
<	Less than
>=	Greater or equal
<=	Less or equal

🧪 Example:

```
#include <stdio.h>

int main() {
    int a = 5, b = 10;

    printf("%d\n", a == b);  // 0 (false)
    printf("%d\n", a != b);  // 1 (true)
    printf("%d\n", a < b);   // 1
    printf("%d\n", a > b);   // 0

    return 0;
}
```

👉 **1 = True, 0 = False**

🔗 11. Logical Operators

📖 Used to combine conditions

Operator	Meaning
&&	AND
`	
!	NOT

🧠 Real-Life Example:

👉 You pass exam if:

- marks > 40 **AND** attendance > 75

🧪 Program:

```
#include <stdio.h>
```

```c
int main() {
    int marks = 50, attendance = 80;

    printf("%d\n", (marks > 40 && attendance > 75)); // 1
    printf("%d\n", (marks > 60 || attendance > 75)); // 1
    printf("%d\n", !(marks > 40)); // 0

    return 0;
}
```

📝 12. Assignment Operator (=)

📖 Used to assign value

```c
int a;
a = 10;
```

👉 Means: store 10 in variable `a`

🧪 Example:

```c
#include <stdio.h>

int main() {
    int a;
    a = 5;

    printf("Value: %d", a);

    return 0;
}
```

⚠️ Important:

👉 = is NOT equal
👉 == is equal comparison

🔄 13. Increment and Decrement

📖 Used to increase/decrease value by 1

Operator	Meaning
++	Increase by 1
--	Decrease by 1

🧪 Example:

```
#include <stdio.h>

int main() {
    int a = 5;

    a++;  // a = 6
    printf("%d\n", a);

    a--;  // a = 5
    printf("%d", a);

    return 0;
}
```

🔥 Pre vs Post (Simple idea)

```
int a = 5;

printf("%d\n", a++); // prints 5, then becomes 6
printf("%d\n", ++a); // becomes 7, then prints 7
```

Control Flow Statements in C

👉 Control flow = **how the program makes decisions and repeats actions**

🔀 14. if, if-else, else-if, nested if

🟢 `if` Statement

👉 Used to check a condition

```
#include <stdio.h>

int main() {
    int age = 18;

    if (age >= 18) {
        printf("You can vote");
    }

    return 0;
}
```

🟡 `if-else`

```
if (age >= 18) {
    printf("Adult");
} else {
    printf("Minor");
}
```

🔵 `else-if` Ladder

```
int marks = 85;

if (marks >= 90) {
    printf("Grade A");
}
else if (marks >= 75) {
    printf("Grade B");
}
else {
    printf("Grade C");
}
```

🔴 Nested if (if inside if)

```
int age = 20;
int id = 1;

if (age >= 18) {
    if (id == 1) {
        printf("Allowed");
    }
}
```

🔘 15. Switch Statement

👉 Used when you have **many options**

🧪 Example:

```
#include <stdio.h>

int main() {
    int day = 2;

    switch(day) {
        case 1:
            printf("Monday");
            break;
        case 2:
            printf("Tuesday");
            break;
        case 3:
            printf("Wednesday");
            break;
        default:
            printf("Invalid");
    }

    return 0;
}
```

⚠️ Important:

- `break` stops execution
- `default` runs if no match

🔁 16. For Loop

👉 Used when you know **how many times to repeat**

🧪 Example:

```
#include <stdio.h>

int main() {
    int i;

    for (i = 1; i <= 5; i++) {
        printf("%d\n", i);
    }

    return 0;
}
```

🔍 Output:

```
1
2
3
4
5
```

🔁 17. While Loop

👉 Used when condition is checked **before running**

🧪 Example:

```
#include <stdio.h>

int main() {
    int i = 1;

    while (i <= 5) {
        printf("%d\n", i);
        i++;
    }

    return 0;
}
```

🔁 18. do-While Loop

👉 Runs **at least one time** (even if condition is false)

🧪 Example:

```
#include <stdio.h>

int main() {
    int i = 1;

    do {
        printf("%d\n", i);
        i++;
    } while (i <= 5);

    return 0;
}
```

⛔ 19. break, continue, goto

🔴 `break`

👉 Stops loop immediately

```
for (int i = 1; i <= 5; i++) {
    if (i == 3)
        break;

    printf("%d\n", i);
}
```

👉 Output:

```
1
2
```

🟡 `continue`

👉 Skips current step

```
for (int i = 1; i <= 5; i++) {
    if (i == 3)
        continue;

    printf("%d\n", i);
}
```

👉 Output:

```
1
2
4
5
```

🔵 `goto` (rarely used)

```
#include <stdio.h>

int main() {
    int i = 1;

start:
    printf("%d\n", i);
    i++;

    if (i <= 3)
        goto start;

    return 0;
}
```

⚠️ Note:

👉 Avoid `goto` in real programs (not recommended)

Summary

- `if` → decision making
- `switch` → multiple choices
- `for` → fixed loop
- `while` → condition-based loop
- `do-while` → runs at least once
- `break` → stop loop
- `continue` → skip step
- `goto` → jump (avoid)

Arrays and Strings in C

📦 20. Arrays (1D & Multi-Dimensional)

📖 What is an Array?

👉 Array = **collection of same type values stored together**

👉 Example: storing marks of 5 students

🟢 One-Dimensional Array (1D)

✍️ Declaration:

```
int marks[5];
```

👉 Can store 5 numbers

✍️ Initialization:

```
int marks[5] = {90, 85, 78, 88, 92};
```

🧪 Example Program:

```
#include <stdio.h>

int main() {
    int marks[5] = {90, 80, 70, 60, 50};

    for (int i = 0; i < 5; i++) {
        printf("%d\n", marks[i]);
    }

    return 0;
}
```

🧠 Easy Idea:

👉 `marks[0] = 90`
👉 `marks[1] = 80`

(Index starts from **0**)

🔵 Multi-Dimensional Array (2D)

👉 Looks like a **table (rows & columns)**

✍️ Example:

```
int matrix[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
```

🧪 Program:

```
#include <stdio.h>

int main() {
    int a[2][2] = {{1,2},{3,4}};

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            printf("%d ", a[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

🔤 21. String Handling (`<string.h>`)

📖 What is a String?

👉 String = **array of characters**

Example:

```
char name[] = "Hello";
```

👉 Stored as: `H e l l o \0`
👉 `\0` = end of string

📌 Header File:

```
#include <string.h>
```

👉 Used for string functions

🧪 Simple String Program:

```
#include <stdio.h>

int main() {
    char name[10];

    printf("Enter your name: ");
    scanf("%s", name);

    printf("Hello %s", name);

    return 0;
}
```

📜 22. Array of Strings

👉 Multiple words (like list of names)

🧪 Example:

```
#include <stdio.h>

int main() {
    char names[3][10] = {"Ram", "Sam", "Ravi"};

    for (int i = 0; i < 3; i++) {
        printf("%s\n", names[i]);
    }

    return 0;
}
```

🧠 Idea:

👉 3 names, each can store 10 characters

🛠️ 23. Common String Functions

📖 Important Functions

🔷 1. `strlen()` → length of string

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[] = "Hello";

    printf("%d", strlen(str));

    return 0;
}
```

👉 Output: `5`

🔷 2. `strcpy()` → copy string

```
char a[10] = "Hi";
char b[10];

strcpy(b, a);
```

👉 Now `b = "Hi"`

🔷 3. `strcat()` → join strings

```
char a[20] = "Hello ";
char b[] = "World";

strcat(a, b);
```

👉 Output: `"Hello World"`

🔷 4. `strcmp()` → compare strings

```
strcmp("abc", "abc"); // 0 (equal)
```

🧪 Full Example:

```
#include <stdio.h>
#include <string.h>

int main() {
    char a[20] = "Hello";
    char b[20] = "World";

    printf("Length: %d\n", strlen(a));

    strcat(a, b);
    printf("Join: %s\n", a);

    printf("Compare: %d", strcmp(a, b));

    return 0;
}
```

- Array = group of values
- 1D array = list
- 2D array = table
- String = array of characters
- Array of strings = list of words
- `<string.h>` = string tools
- Functions:
    - `strlen()` → length
    - `strcpy()` → copy
    - `strcat()` → join
    - `strcmp()` → compare

Functions in C

👉 Function = **a small block of code that does a specific task**

👉 Think like:

- Function = **machine**
- Input → Process → Output

📢 24. Function Declaration & Definition

🟢 Function Declaration (Prototype)

👉 Tells the compiler **what the function will do**

```
int add(int a, int b);
```

🔵 Function Definition

👉 Actual code of the function

```
int add(int a, int b) {
    return a + b;
}
```

🧠 Easy Idea:

- Declaration = **telling**
- Definition = **doing**

📞 25. Calling a Function

👉 Calling = **using the function**

🧪 Example:

```c
#include <stdio.h>

int add(int a, int b);  // declaration

int main() {
    int result;

    result = add(5, 3);   // function call
    printf("Sum: %d", result);

    return 0;
}

int add(int a, int b) {  // definition
    return a + b;
}
```

🔄 26. Function Arguments & Return Values

📖 Arguments (Inputs)

```c
add(5, 3);
```

👉 5 and 3 are arguments

📖 Return Value (Output)

```c
return a + b;
```

👉 Function gives back result

🧪 Example:

```c
#include <stdio.h>

int square(int x) {
    return x * x;
}

int main() {
    int result = square(4);

    printf("Square: %d", result);

    return 0;
}
```

🔁 27. Call by Value & Call by Reference

🟢 Call by Value

👉 Copy of value is passed
👉 Original value **not changed**

```
#include <stdio.h>

void change(int x) {
    x = 100;
}

int main() {
    int a = 10;

    change(a);
    printf("%d", a);  // still 10

    return 0;
}
```

🔵 Call by Reference

👉 Address is passed
👉 Original value **changes**

```
#include <stdio.h>

void change(int *x) {
    *x = 100;
}

int main() {
    int a = 10;

    change(&a);
    printf("%d", a);  // 100

    return 0;
}
```

🧠 Simple Idea:

- Value → copy 📄
- Reference → original 📦

🔁 28. Recursion

📖 What is Recursion?

👉 Function calling **itself**

⚠️ Important:

👉 Must have **base condition** (stop condition)

🧪 Example: Factorial

👉 5! = 5 × 4 × 3 × 2 × 1

```
#include <stdio.h>

int fact(int n) {
    if (n == 1)   // base condition
        return 1;

    return n * fact(n - 1);
}

int main() {
    int result = fact(5);

    printf("Factorial: %d", result);

    return 0;
}
```

🔍 Output:

```
Factorial: 120
```

🧠 Simple Understanding:

```
fact(5)
= 5 × fact(4)
= 5 × 4 × fact(3)
...
= 120
```

🎯 Final Simple Summary

- Function = reusable code block
- Declaration = tell function
- Definition = write function
- Calling = use function
- Arguments = inputs
- Return = output
- Call by Value = no change
- Call by Reference = changes original
- Recursion = function calls itself

🎓 Practice for You

```
#include <stdio.h>

int cube(int x) {
    return x * x * x;
}

int main() {
    printf("%d", cube(3));
    return 0;
}
```